

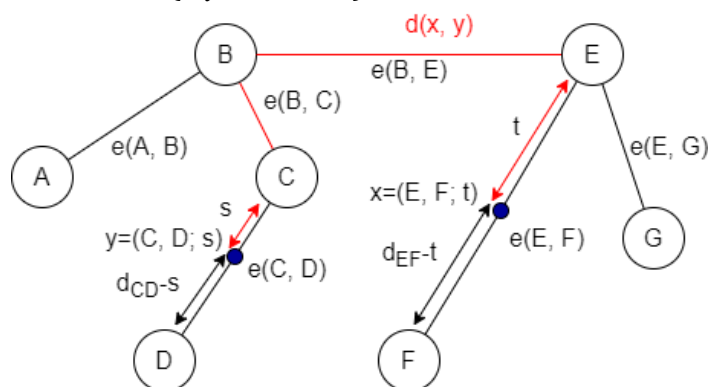
Linear-Time Algorithm for Linear Programming in R^3 and Related Problems. The weighted center of a tree.

Society for Industrial and Applied Mathematics, Vol. 12, No. 4, November 1983.

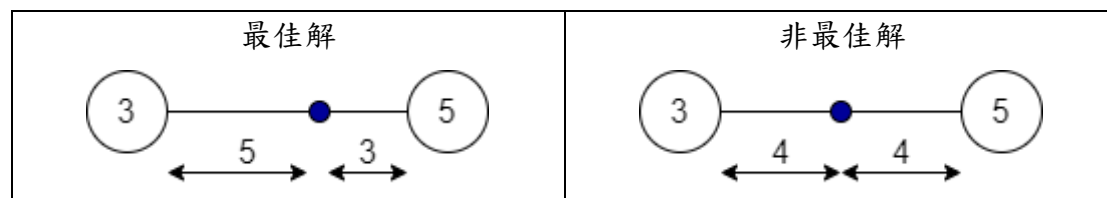
問題定義

此演算法的目的是要找到一個 Tree 的唯一一點，使得這點到任何一點的距離的最大值達到最小。

假設 Tree $T = (V, E)$ ， w_i 為節點的權重， d_{ij} 表示 $e(i, j)$ 的距離， $x=(i, j; t)$ 表示 x 點在 i 往 j 距離為 t 的地方， $d(x, y)$ 則是這兩點的最短路徑距離；此演算法要找到的是 $r(x) = \max\{w_i d(x, i) : i \in V\}$ 的最小值



以下圖為例，解會較靠近權重較重的節點，使得兩邊的距離和權重相乘均為 15，即 $r(x)=15$ ；選擇不為解的點，必會有節點使 $r(x) > 15$ 。



以三個節點為例，解不一定會使每個節點的距離乘以權重相等，但其最大值必會使 $r(x)$ 為最小，其它的點都會使 $r(x) > 15$ 。

最佳解	非最佳解
$3 \times 5 = 15$	$3 \times 4 = 12$
$5 \times 3 = 15$	$5 \times 4 = 20$
$1 \times 9 = 9$	$1 \times 10 = 10$

此演算法就是要找到這唯一的點。

解法敘述

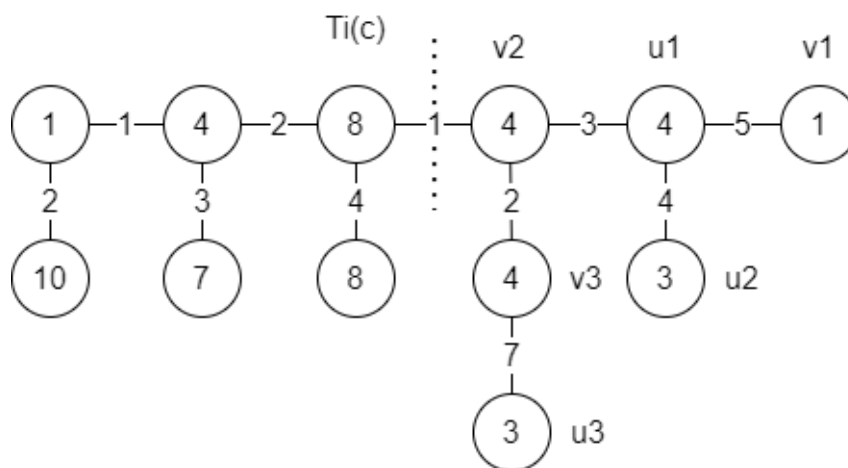
以兩三個點的 Tree 來說，我們可以簡單地用直覺觀察節點的權重及邊緣的距離，猜測最佳解落在哪個地方，並且暴力計算出兩側的最大值，當兩者相等的时候就是最佳解的位置；因為這時往哪側移動，都會使其中一邊的值變大。

以下圖一為例，當我們不知道解在什麼地方，我們可以先假設在任一個點，這邊假設在左邊線的中間，這時候會發現右側的數值比較大，而將點點往右移動時，由右側的數值會變小。而圖二假設最佳解在右邊線上，從數值可以看出，最佳解應該往左移動，使左子樹的值變大，右子樹的值變小。

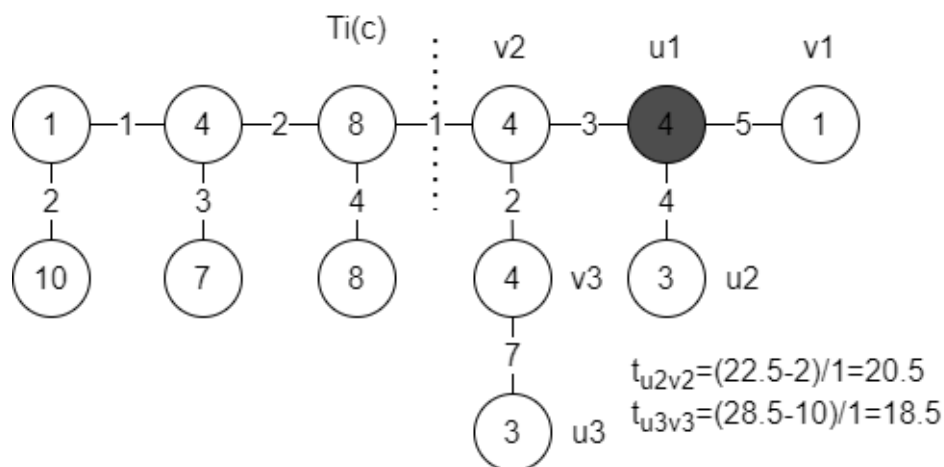
圖一	左側	右側
	$2 \times 1 = \underline{2}$	$8 \times 1 = 8$
		$10 \times 3 = \underline{30}$
		$4 \times 3 = 12$
圖二	左側	右側
	$2 \times 3 = 6$	$10 \times 1 = \underline{10}$
	$8 \times 1 = 8$	
	$4 \times 3 = \underline{12}$	

也就是說，我們可以先排除一半的節點，知道最佳解在哪個子樹。接著我們可以放棄一些不影響結果的節點。

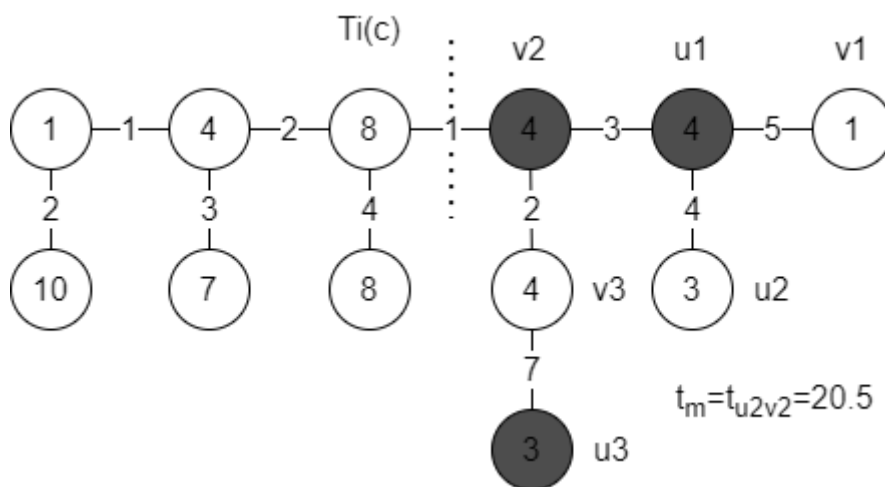
假設最佳解 c 在子樹 $T_i(c)$ 中，將剩下的節點兩兩配對且路徑不重疊（如果有多出來的先不管它），每一對 (u, v) 使得 $w_u d(u, c) \geq w_v d(v, c)$ 。以下圖為例， $T_i(c)$ 在左側，將右側的四個點兩兩配對，並且決定誰是 u 誰是 v 。



若 $w_u \geq w_v$ ，則可以放棄考慮 w_v ，因為無論解移動到目前邊線的哪個位置， v 都無法影響結果。 $w_u < w_v$ 的配對則計算 (u,v) 間的重心 $t_{uv} = (w_u d(u, c) - w_v d(v, c)) / (w_v - w_u)$ 。



找出這些 t_{uv} 的中位數 t_m 。從 $t_{uv} \geq t_m$ 的這些 (u,v) 中，因為 $w_u d(u, c) \geq w_v d(v, c)$ ，可以放棄考慮 w_v ； $t_{uv} < t_m$ 的配對中，則可以放棄考慮 w_u 。



我們在最一開始判斷最佳解在哪一個子樹時，選擇的點使得兩側節點的數量 $|V_j(x)| \leq n/2$ ，接著又將非最佳解的子樹的節點兩兩配對，並且放棄其中一半的節點，因此每一輪操作能少掉 $1/8$ 的節點 ($T(n) \leq T(\frac{7n}{8}) + Cn = O(n)$)，最後會剩下兩個節點，就能算出這兩點間的重心，因而找到最佳解的位置。

